



FACULTY OF ELECTRONICS AND COMPUTER
TECHNOLOGY AND ENGINEERING

SEM II - 2025/26

BERR 2243

DATABASE AND CLOUD SYSTEM

EXERCISE 6 REPORT

LECTURER NAME:

PROFESOR MADYA DR SOO YEW GUAN

No.	Student Name	Matrix No.
1	IKMAL IMAN BIN MOHD YUSOF	B122410306
2	MUHAMMAD ARIF SYAFIE BIN AGUS SALEH	B022210116
3	MUHAMMAD HAIKAL BIN ZULKIFLI	B122410348

Table of Contents

1. Objective.....	1
2. Methodology & Setup Procedure.....	1
a. Deploying MongoDB via Docker.....	1
b. Database Verification via MongoDB Compass.....	2
c. Installing Driver Nodes in Node-RED.....	2
d. Configuring the Database Connection Link.....	3
3. Results & Observations.....	3
a. Part A: CREATE (POST /api/sensor).....	3
b. Part B: READ (HTTP GET /api/telemetry).....	6
c. Part C: UPDATE (HTTP PUT /api/sensor).....	8
d. Part D: DELETE (HTTP DELETE /api/sensor).....	10
4. Discussion & Architectural Analysis.....	13
Question 1: Dynamic Schema Generation.....	13
Question 2: Middleware Abstraction.....	13
5. Conclusion.....	14

1. Objective

The primary objectives of this laboratory exercise are:

- To set up and run a local MongoDB database on my computer using Docker.
- To connect Node-RED to MongoDB, install the required driver.
- To upgrade my existing HTTP API endpoints to support CRUD operations (create, read, update, delete) directly in the database.

2. Methodology & Setup Procedure

a. Deploying MongoDB via Docker

Docker Desktop was used to search for and pull the official **mongodb/mongodb-community-server** image. A new container named **MongoDB** was launched, mapping the database's default port **27017** to the host machine to allow external connections.

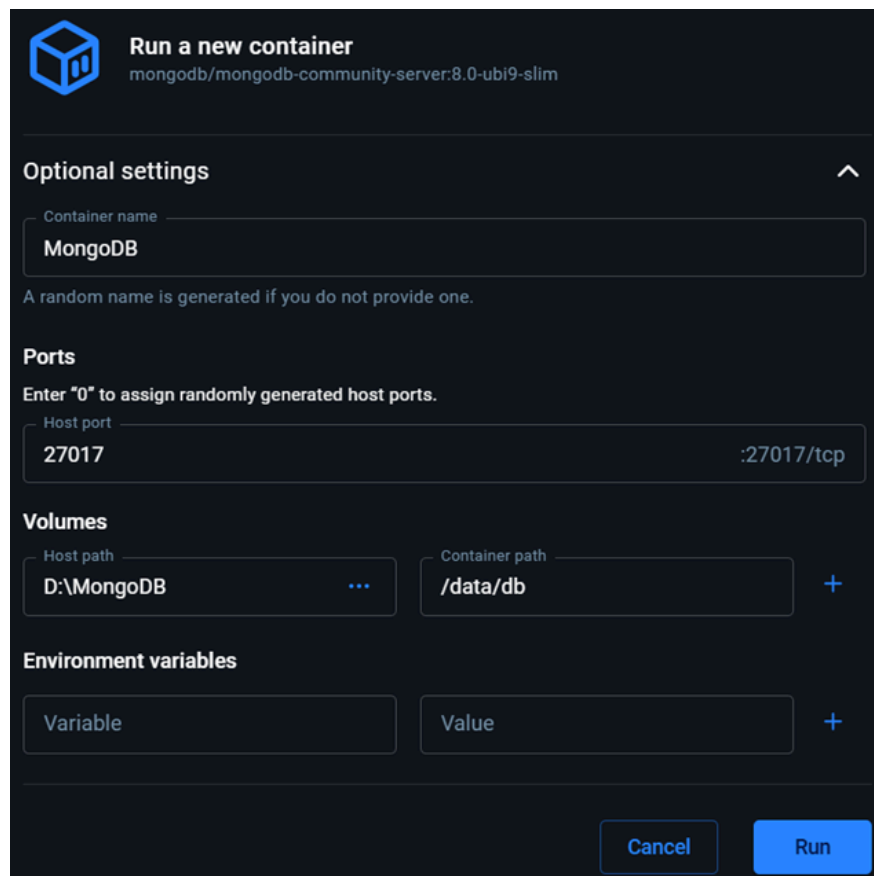


Fig. 2a. 1. MongoDB container in Docker Desktop

b. Database Verification via MongoDB Compass

To confirm the database was running properly, MongoDB Compass was opened and connected using the local connection string **mongodb://localhost:27017**. This allowed visual management of the database collections.

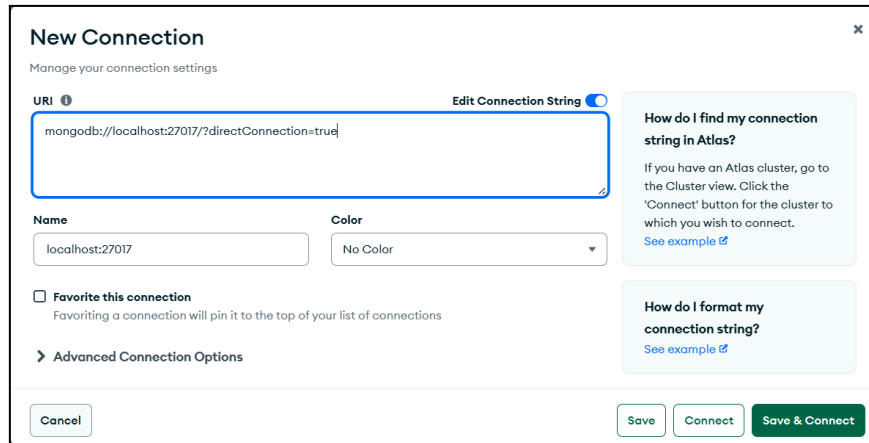


Fig. 2b. 1. Connection window in MongoDB Compass

c. Installing Driver Nodes in Node-RED

Inside Node-RED's Palette Manager, the **node-red-contrib-mongodb4** driver package was searched and installed. This added the native **mongodb4** storage nodes to the Node-RED sidebar so the flow could communicate with the database.

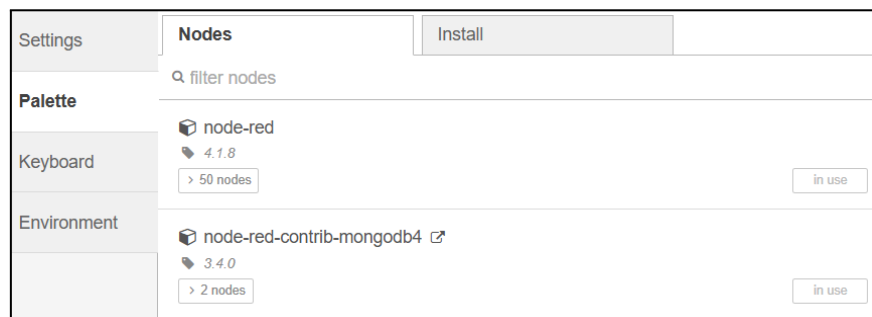
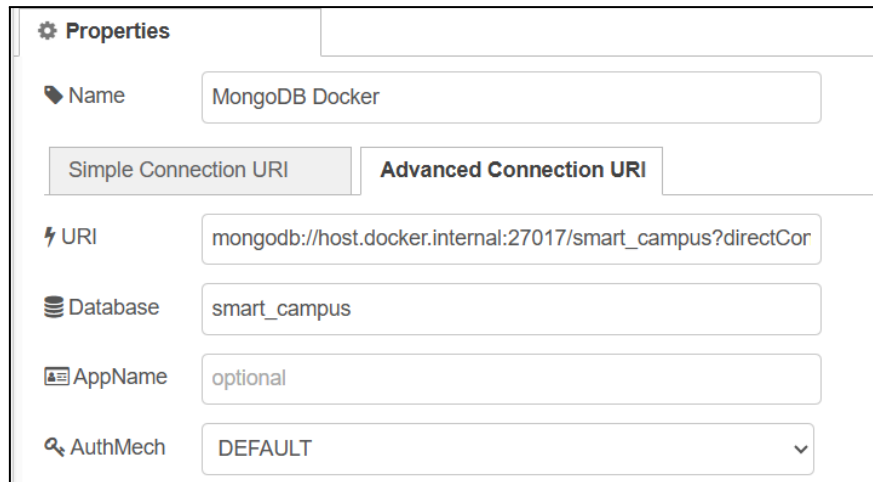


Fig. 2c. 1. Palette Manager showing the installed mongodb4 package

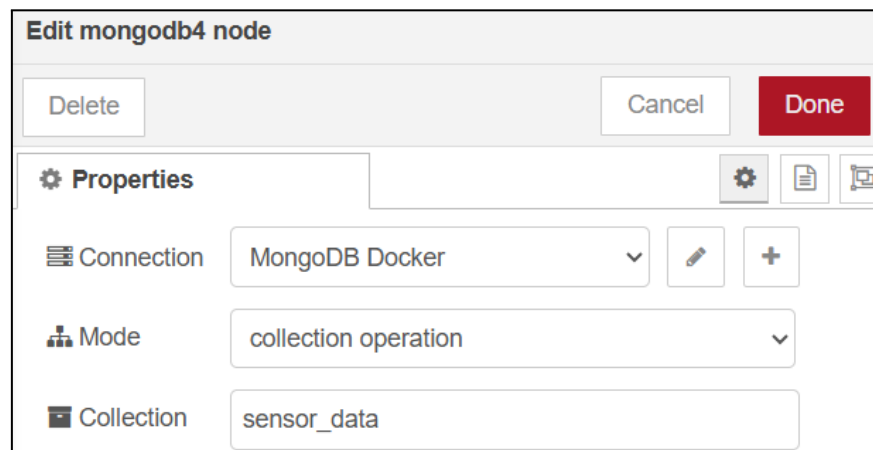
d. Configuring the Database Connection Link

A **mongodb4** node was dragged onto the workspace. Inside its settings, a new server connection configuration was added with the connection URI set to `mongodb://host.docker.internal:27017/smart_campus?directConnection=true`. The target database was named **smart_campus**, and the collection was set to **sensor_data**.



The screenshot shows a 'Properties' dialog for a 'MongoDB Docker' connection. It has two tabs: 'Simple Connection URI' and 'Advanced Connection URI'. The 'Advanced Connection URI' tab is selected. The fields are: 'URI' (mongodb://host.docker.internal:27017/smart_campus?directCor), 'Database' (smart_campus), 'AppName' (optional), and 'AuthMech' (DEFAULT).

Fig. 2d. 1. mongodb4-client URI configuration



The screenshot shows an 'Edit mongodb4 node' dialog. It has a 'Delete' button, a 'Cancel' button, and a 'Done' button. Below these are three tabs: 'Properties', 'Connection', and 'Mode'. The 'Properties' tab is selected. The fields are: 'Connection' (MongoDB Docker), 'Mode' (collection operation), and 'Collection' (sensor_data).

Fig. 2d. 2. mongodb4-client node configuration

3. Results & Observations

a. Part A: CREATE (POST /api/sensor)

To allow edge devices to send and store sensor data, an HTTP POST endpoint was created at the URL **/api/sensor**. The incoming request body is passed directly into a **mongodb4** node configured with the **insertOne** operation to save the data into the **sensor_data** collection.

Edit http in node

Delete Cancel Done

Properties

Method POST

☐ Do not parse request body

☐ Accept file uploads

URL /api/sensor

Name POST Node

Fig. 3a. 1. Properties of the POST Node

Edit mongodb4 node

Delete Cancel Done

Properties

Connection MongoDB Docker

Mode collection operation

Collection sensor_data

Operation insertOne

Output toArray

MaxTimeMS 0

☐ Handle document _id (deprecated)

Name POST (insertOne)

Fig. 3a. 2. Properties of the mongodb4 insertOne node

To test this endpoint, a POST request containing a sample JSON payload (**sensor_id**, **temperature**, **humidity**, and **status**) was sent via Postman. The API successfully returned a **200 OK** status alongside a database confirmation showing **acknowledged: true** and the unique **insertedId**.

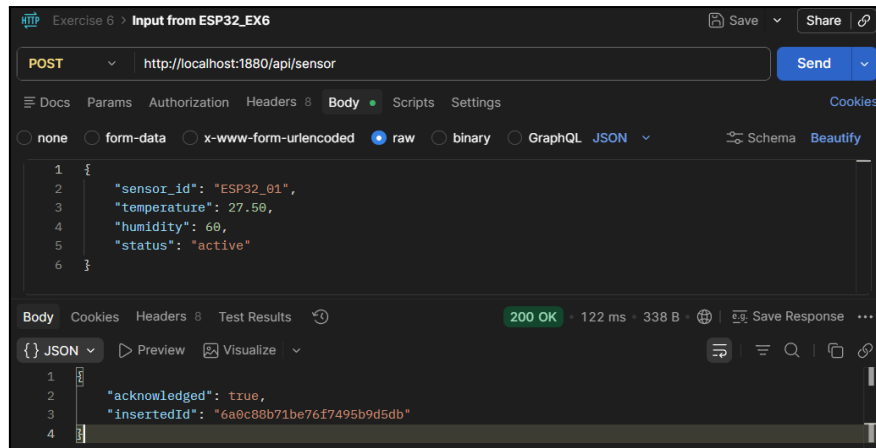


Fig. 3a. 3. Postman successful POST response

Finally, MongoDB Compass was checked to verify data storage. The document was successfully registered with the identical unique `_id` string generated during the Postman request, proving that the data is stored permanently.

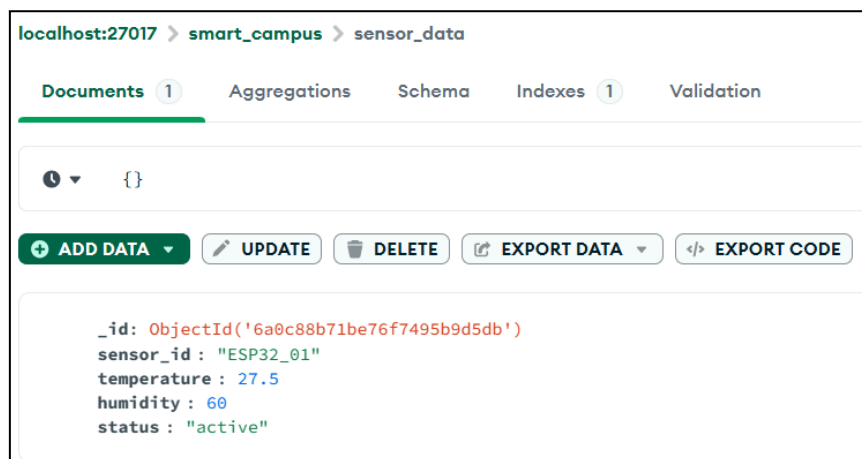
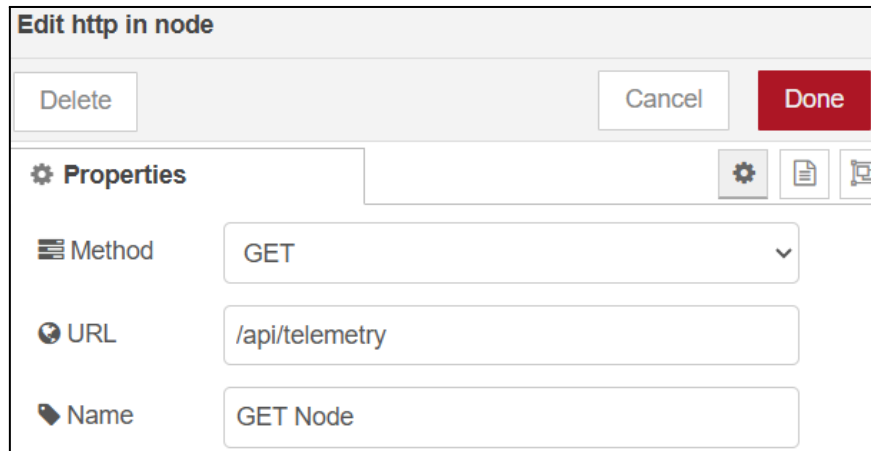


Fig. 3a. 4. Saved document in MongoDB Compass

b. Part B: READ (HTTP GET /api/telemetry)

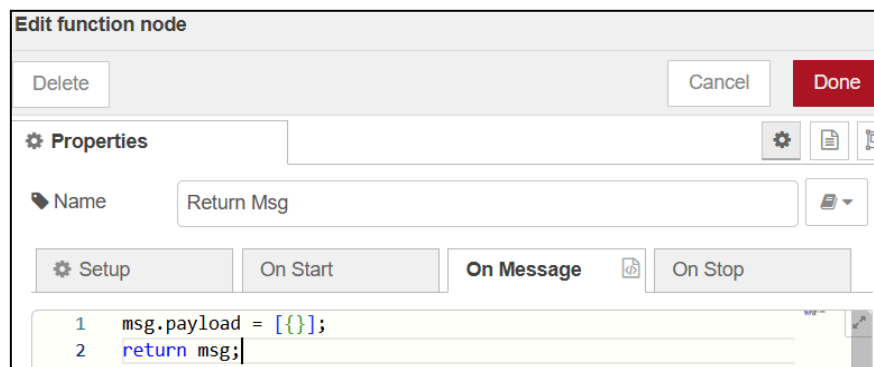
To retrieve the saved sensor data from the database, an HTTP GET endpoint was created at the URL **/api/telemetry**.



The screenshot shows the 'Edit http in node' dialog box. It has a title bar with the text 'Edit http in node'. Below the title bar are three buttons: 'Delete', 'Cancel', and 'Done'. Below these buttons is a 'Properties' section with a gear icon and three fields: 'Method' set to 'GET', 'URL' set to '/api/telemetry', and 'Name' set to 'GET Node'.

Fig. 3b. 1. Properties of the GET Node

A Function node was placed right after the GET node. Inside this node, the payload was set to **msg.payload = [{}]**. This empty object serves as a blank filter, telling MongoDB to pull all files from the storage without filtering anything out.



The screenshot shows the 'Edit function node' dialog box. It has a title bar with the text 'Edit function node'. Below the title bar are three buttons: 'Delete', 'Cancel', and 'Done'. Below these buttons is a 'Properties' section with a gear icon and a 'Name' field set to 'Return Msg'. Below the 'Properties' section are four tabs: 'Setup', 'On Start', 'On Message', and 'On Stop'. The 'On Message' tab is selected, showing a code editor with two lines of JavaScript code: '1 msg.payload = [{}];' and '2 return msg;'.

Fig. 3b. 2. Properties of the function node

This request is then passed into a **mongodb4** node. The node's operation is set to **find**, and the output is set to **toArray**. This configuration converts the raw database results into a standard list format that applications can easily read.

Edit mongodb4 node

Delete Cancel Done

Properties

Connection: MongoDB Docker

Mode: collection operation

Collection: sensor_data

Operation: find

Output: toArray

MaxTimeMS: 0

☐ Handle document _id (deprecated)

Name: GET (find)

Fig. 3b. 3. Properties of the mongodb4 find node

To test the endpoint, a GET request was sent to the URL using Postman. The API successfully returned a **200 OK** status along with a JSON array containing the stored sensor data, including its unique `_id`.

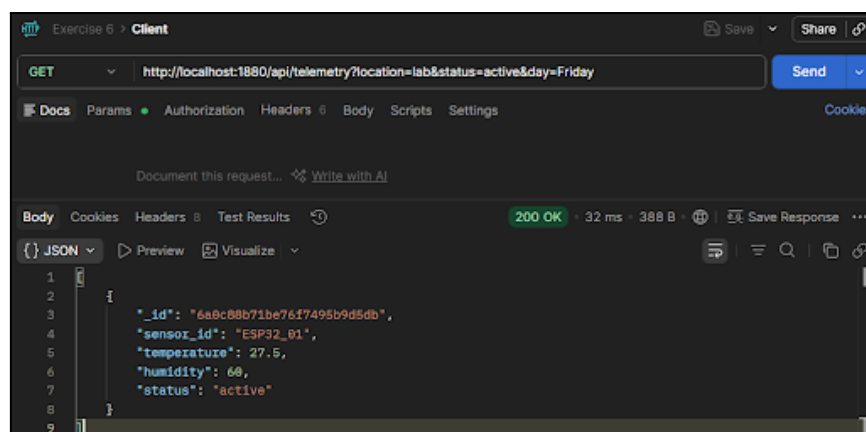
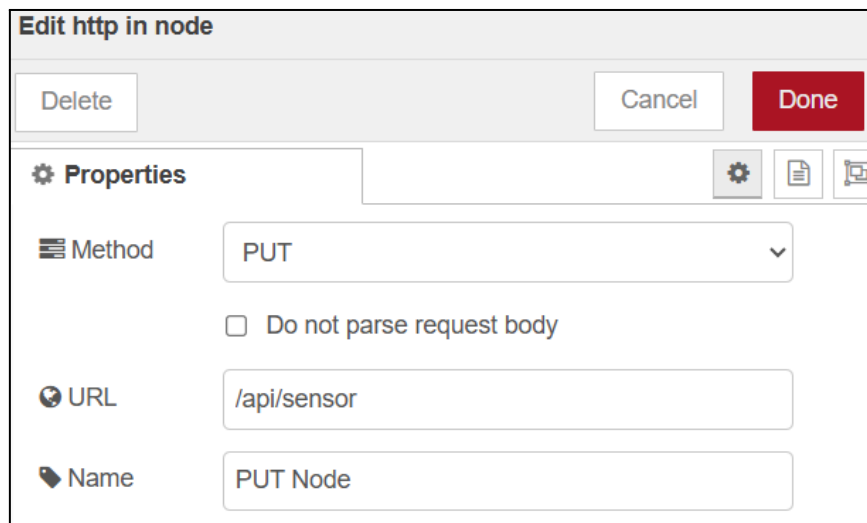


Fig. 3b. 4. Postman successful GET response

c. Part C: UPDATE (HTTP PUT /api/sensor)

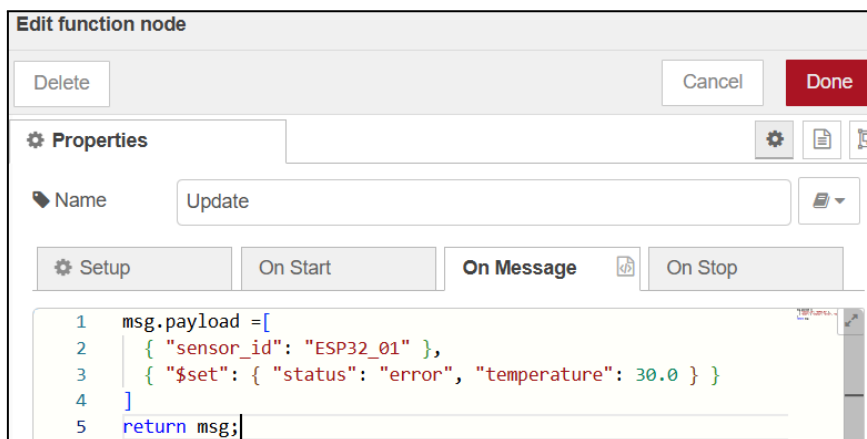
To modify existing data in the database, an HTTP PUT endpoint was created at the URL `/api/sensor`.



The screenshot shows the 'Edit http in node' dialog box. It has a title bar 'Edit http in node' and three buttons: 'Delete', 'Cancel', and 'Done'. Below the buttons is a 'Properties' section with a gear icon and three sub-sections: 'Method' with a dropdown menu set to 'PUT', 'Do not parse request body' with an unchecked checkbox, 'URL' with a text field containing '/api/sensor', and 'Name' with a text field containing 'PUT Node'.

Fig. 3c. 1. Properties of the PUT Node

A Function node was connected after the PUT node to format the update instruction as a two-element JavaScript array required by the database driver. The first element defines the filter to find the specific device (`"sensor_id": "ESP32_01"`), and the second element uses the `$set` operator to update its `status` to `"error"` and its `temperature` to `30.0`.



The screenshot shows the 'Edit function node' dialog box. It has a title bar 'Edit function node' and three buttons: 'Delete', 'Cancel', and 'Done'. Below the buttons is a 'Properties' section with a gear icon and a 'Name' text field containing 'Update'. Below the 'Name' field are four tabs: 'Setup', 'On Start', 'On Message', and 'On Stop'. The 'On Message' tab is selected, showing a JavaScript code editor with the following code:

```
1 msg.payload = [  
2   { "sensor_id": "ESP32_01" },  
3   { "$set": { "status": "error", "temperature": 30.0 } }  
4 ]  
5 return msg;
```

Fig. 3c. 2. Properties of the update function node

This payload is then passed into a `mongodb4` node configured with the `updateOne` operation targeting the `sensor_data` collection.

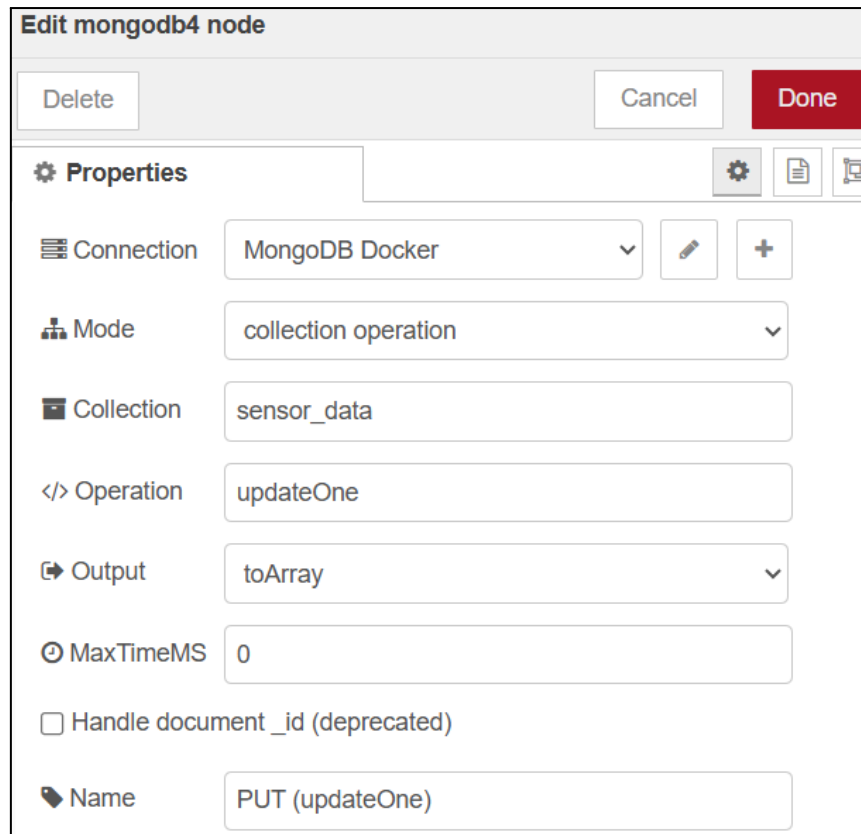


Fig. 3c. 3. Properties of the mongodb4 updateOne node

To test the update logic, a PUT request was triggered from Postman. The endpoint successfully returned a **200 OK** status with a response body showing **matchedCount: 1** and **modifiedCount: 1**, confirming that the database located the target record and applied the changes.

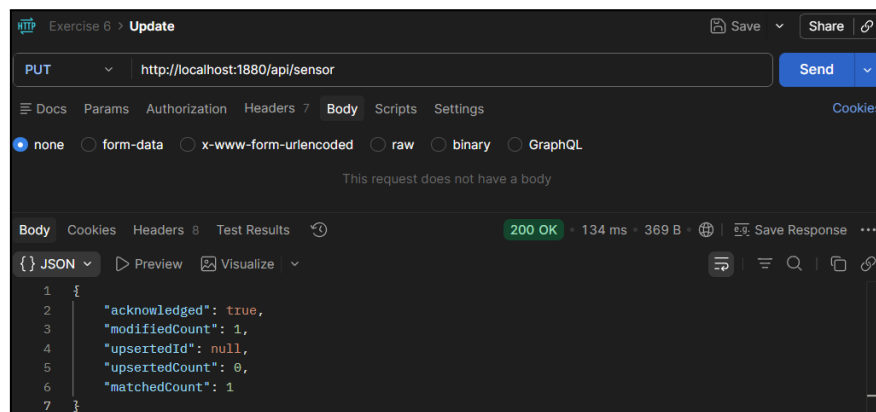


Fig. 3c. 4. Postman successful PUT response

Finally, checking MongoDB Compass confirmed that the document with **sensor_id: "ESP32_01"** had its temperature successfully changed to **30**, and its status changed to **"error"**.

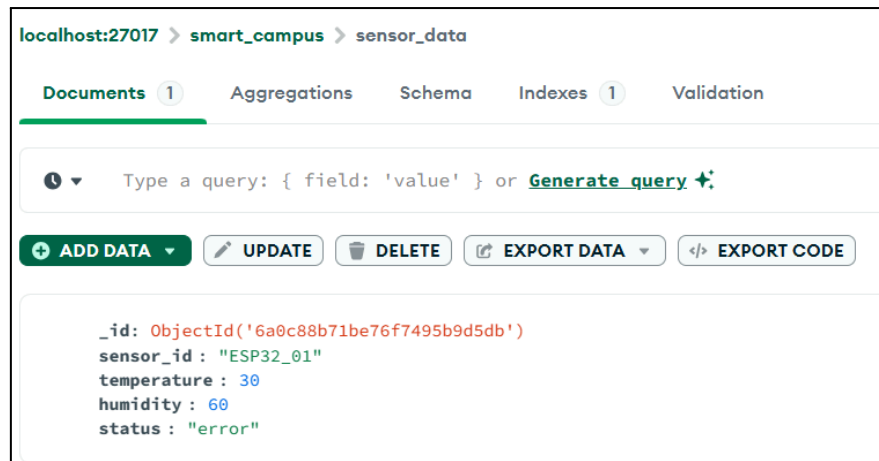


Fig. 3c. 5. Updated document in MongoDB Compass

d. Part D: DELETE (HTTP DELETE /api/sensor)

To remove data from the database, an HTTP DELETE endpoint was created at the URL `/api/sensor`.

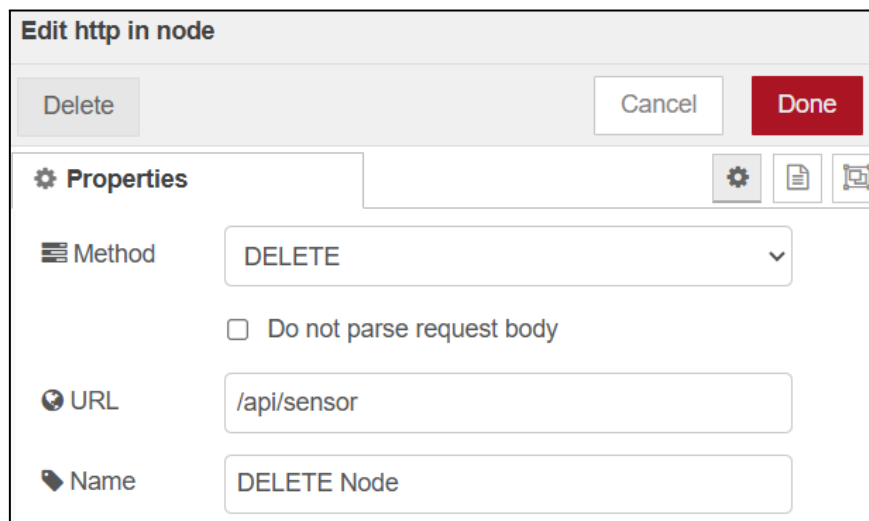


Fig. 3d. 1. Properties of the DELETE node

A function node was placed after the DELETE node to define which records should be removed. Following the exercise instructions, the payload was structured as a single-element array containing the filter `{ "status": "error" }` to target any data entries marked with an error status.

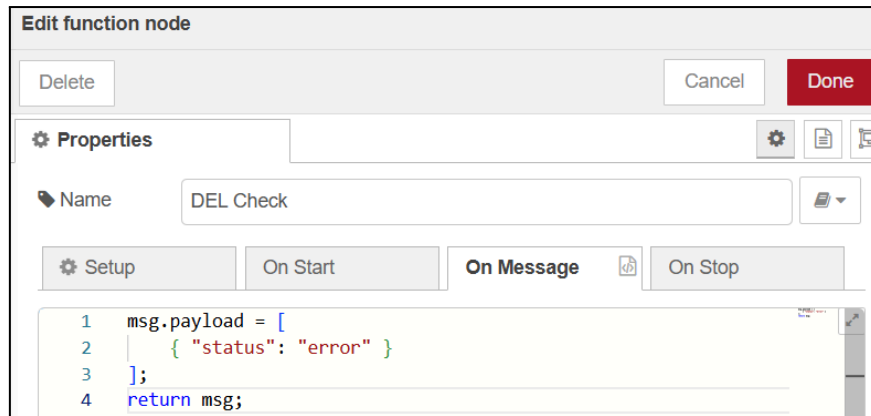


Fig. 3d. 2. Properties of the DELETE function node

This filter payload is sent into a **mongodb4** node, where the operation is configured to **deleteMany**. This allows the system to wipe out all documents matching the error criteria inside the **sensor_data** collection at once.

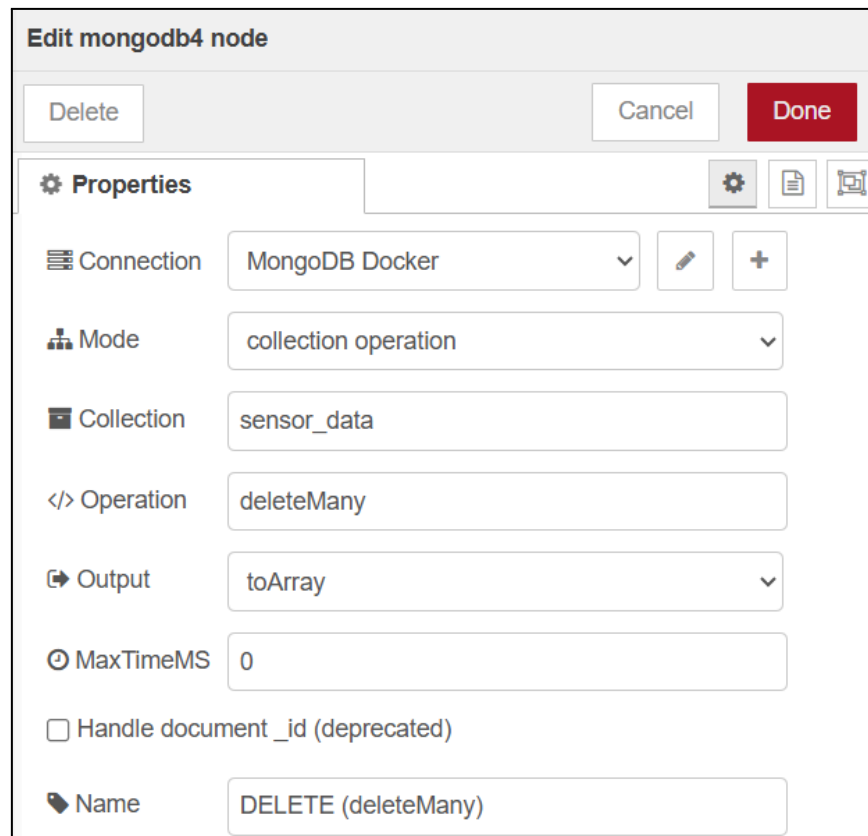


Fig. 3d. 3. Properties of the mongodb4 deleteMany node

To test the deletion process, a DELETE request was sent from Postman to the API URL. The database responded with a **200 OK** status and a body showing **acknowledged: true** and **deletedCount: 1**, confirming that the record with the error status was successfully erased.

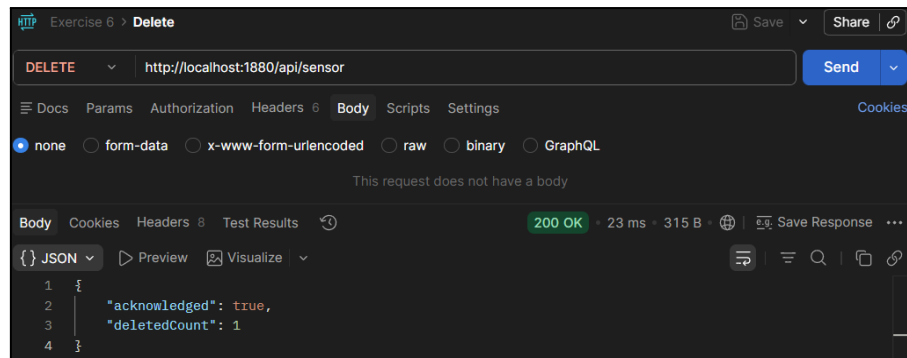


Fig. 3d. 4. Postman successful DELETE response

Finally, checking MongoDB Compass confirmed that the operation was fully executed. Because the target document was the only file left in the database, the **sensor_data** collection is now empty, proving the permanent removal of the data.

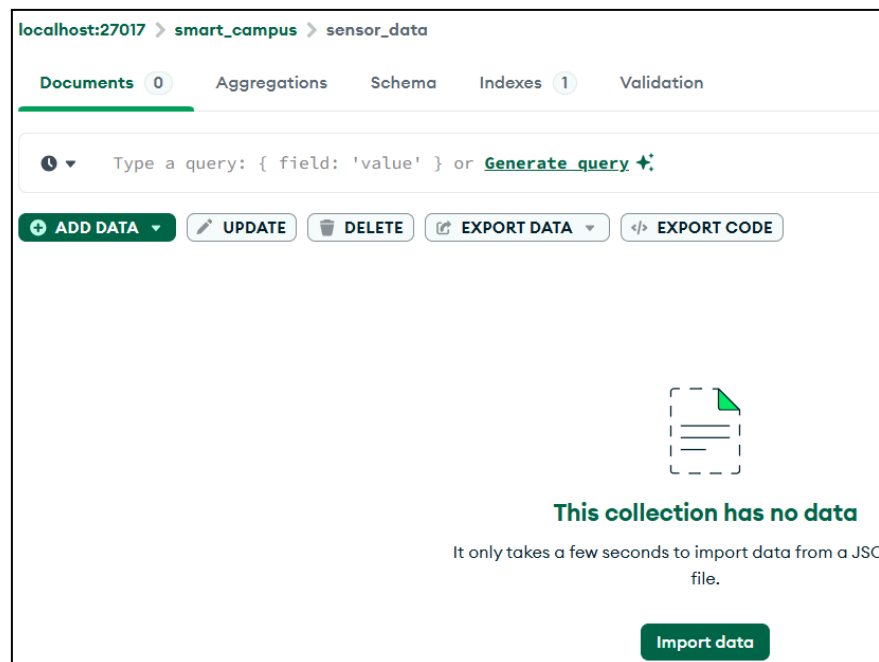


Fig. 3d. 5. Empty collection in MongoDB Compass

4. Discussion & Architectural Analysis

Question 1: Dynamic Schema Generation

When you executed the POST request in Step 4A, you did not have to write a complex SQL CREATE TABLE query beforehand to define columns for "temperature" or "humidity". Explain the concept of a "Schema-less" database. Why is this specific feature highly beneficial during the early prototyping phases of an IoT network, where sensor data formats might frequently change?

A schema-less database is a database that does not require fixed tables or predefined columns before saving data. Instead of following strict rules about what type of data can be entered, it automatically accepts and stores information in whatever format it is sent.

This feature is highly beneficial during the early prototyping phases of an IoT network because sensor formats change frequently. If a prototype initially sends only temperature data but is later updated to include humidity, the schema-less database instantly adapts to store the new fields. This allows developers to test and upgrade sensors quickly without having to stop the system or rebuild the database structure every time a change is made.

Question 2: Middleware Abstraction

In this architecture, your ESP32 (simulated by Postman) never talks directly to the MongoDB database. It only talks to Node-RED via HTTP endpoints, and Node-RED handles the database queries. Defend this architecture. Why is it a massive security risk to expose port 27017 directly to the public internet so that edge sensors can write directly to the database?

Using Node-RED as a middle layer creates a controlled and secure bridge between the sensors and the database. Instead of letting devices connect directly to the storage, Node-RED receives the HTTP requests first. This allows the system to filter the data, check for errors, and manage network traffic before anything is actually written to the database.

Exposing port 27017 directly to the public internet is a massive security risk because it is the main entry point for MongoDB. If this port is left open, anyone on the internet can find and access it. Without a protective middle layer like Node-RED, unauthorised users or attackers could connect directly to the database to steal, alter, or completely delete the stored information.

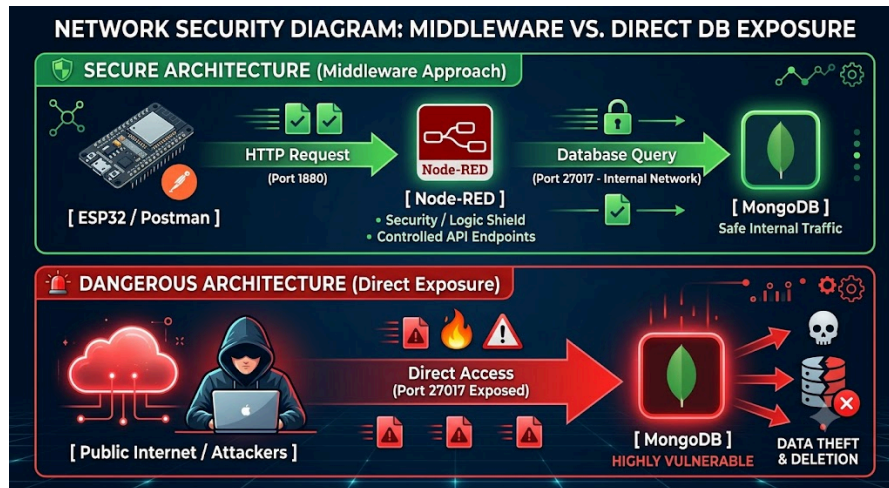


Fig. 4. 1. The Role of Node-RED in Protecting Database Ports

5. Conclusion

In conclusion, this exercise successfully demonstrated how to deploy a local MongoDB database using Docker and connect it to Node-RED. By upgrading the HTTP API endpoints, the system was fully equipped to perform all CRUD operations—creating, reading, updating, and deleting sensor data. Testing with Postman confirmed that the data was reliably saved and managed in permanent storage.

Furthermore, this laboratory exercise highlighted two important system design concepts. First, using a schema-less database provides the flexibility needed to easily change sensor data formats during prototyping. Second, using Node-RED as a middleware creates a secure barrier that protects the database from direct external access. Overall, this exercise provided a strong, practical foundation for securely managing data in an IoT network.